

VITyarthi Calorie Counter

Project Report

Project Type: Python Command-Line Application

Technology: Python 3.10+ Standard Library

Architecture: Modular, service-oriented package with typed models and persistence

Testing: Python unittest

1. Introduction

VITyarthi Calorie Counter is a local command-line application for maintaining a basic user profile, calculating estimated BMR and TDEE/reference values, recording food consumption, and reviewing daily calorie totals. The project was redesigned from a single Python file into a maintainable package.

2. Objectives

The primary objectives are to demonstrate modular Python development, clean separation of concerns, input validation, persistent data storage, exception handling, logging, type hints, and unit testing.

3. Functional Modules

Profile Management: collects profile data and calculates BMR and TDEE/reference values.

Food Management: provides a food database, quantity-based calorie calculations, and custom entries.

Daily Summary: aggregates food entries and compares total consumption with the saved reference value.

4. System Architecture

The application is divided into CLI, services, calculation, validation, models, repository, constants, exceptions, and logging modules. This reduces coupling and allows core components to be tested independently.

```
calorie_counter/  
■■■ cli.py  
■■■ calculator.py  
■■■ service.py  
■■■ models.py  
■■■ repository.py  
■■■ validators.py  
■■■ constants.py  
■■■ exceptions.py  
■■■ logging_config.py  
  
tests/  
■■■ test_calculator.py  
■■■ test_repository.py  
■■■ test_service.py  
■■■ test_validators.py
```

5. Data Persistence

Application data is stored locally as JSON. The repository validates the JSON structure, limits the accepted file size, and uses atomic replacement during saves to reduce the chance of partially written

data.

6. Validation and Error Handling

The application validates text, numeric values, ranges, gender selection, activity levels, food quantities, custom calorie values, and daily entry limits. Dedicated exceptions distinguish validation, storage, and food lookup errors.

7. Testing

The project includes unittest coverage for calculations, validation, persistence, profile creation, food operations, entry limits, and summaries. Tests can be executed with `python -m unittest discover -s tests -v`.

8. Security and Maintainability

The application does not execute user input as Python code, invoke shell commands, or use external network services. Type annotations, focused modules, structured logging, explicit exceptions, and unit tests improve maintainability.

9. Conclusion

The resulting project satisfies the requested academic software structure with multiple meaningful modules, three distinct functional areas, persistent storage, validation, logging, documentation, and automated tests.